

Taller 01 — Foundation Models: Comparación, Decodificación y Razonamiento

Jaime Astudillo y Roberth Chachalo

MMIA 6013 — IA Generativa y Agentes, Universidad San Francisco de Quito

1 Tarea elegida

Elegimos diez problemas de aritmética con respuesta entera para verificar automáticamente los resultados. Agregamos tres casos contaminados y comparamos cómo cambian las respuestas al modificar el modelo, el prompt y el esfuerzo.

Los diez casos limpios y los tres contaminados, con sus respuestas esperadas, están en `datos/casos.json` del repositorio.

2 Parte 0 — El modelo base y la distribución

Fila `base_local`: `openai-community/gpt2`, 124M de parámetros, en CPU, sin API ni credenciales.

2.1 La distribución del siguiente token

En el dominio de números y cálculo usamos `As easy as one, two,` como frase hecha relativamente predecible y `To calculate the total, we need to` como contexto abierto. Los prefijos están en inglés, la lengua predominante del modelo base. La certeza se comprueba con la probabilidad máxima y la entropía; no se supone.

Se hace un forward por prefijo y se reutilizan sus logits para todas las temperaturas. La softmax, la entropía y el núcleo se calculan sobre **todo el vocabulario**, aunque la primera figura solo muestre los 15 primeros tokens.

Al bajar la temperatura, observamos más probabilidad concentrada en los primeros tokens. Al subirla, aumentaron la entropía y las alternativas. Con temperatura 1, el núcleo del 90 % incluyó cinco tokens en el prefijo seguro y 241 en el abierto.

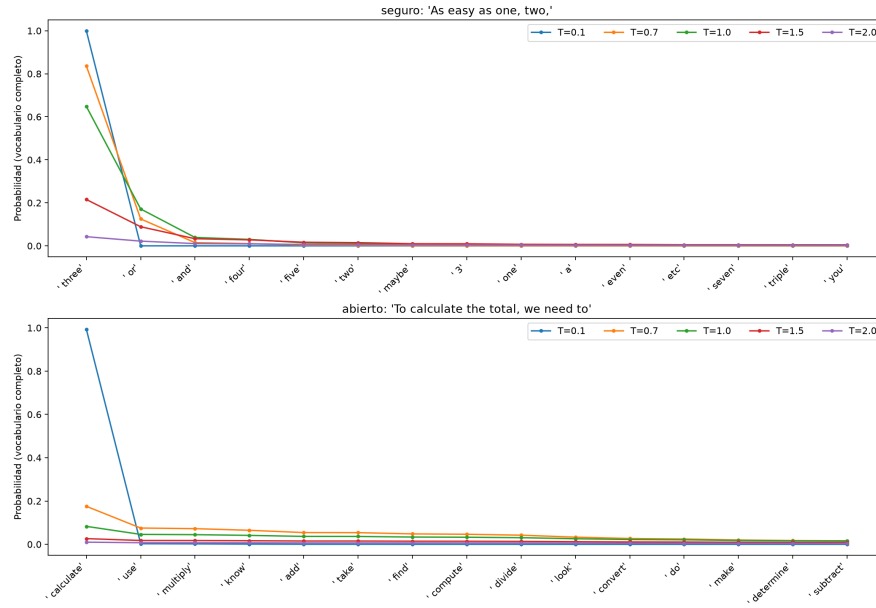


Figura 1. Distribución de los 15 candidatos más probables para $T \in \{0.1, 0.7, 1.0, 1.5, 2.0\}$ en los dos prefijos.

Tabla 1. Entropía y tamaño del núcleo del 90 % por prefijo y temperatura (GPT-2).

prefijo	T	entropía (bits)	núcleo 0.9	p máx
seguro	0.1	3.444e-05	1	1
seguro	0.7	0.8858	2	0.8365
seguro	1	2.323	5	0.6482
seguro	1.5	9.068	7659	0.2155
seguro	2	13.23	20702	0.04256
abierto	0.1	0.0684	1	0.9937
abierto	0.7	5.129	40	0.1755
abierto	1	7.1	241	0.08293
abierto	1.5	9.894	1735	0.02624
abierto	2	11.82	6464	0.009824

2.2 Las tres palancas sobre esa misma distribución

Usamos `model.generate(...)`, modo evaluación y semilla registrada por corrida. Para aislar cada palanca desactivamos los otros filtros: `top_k=0`, `top_p=1`, sin penalización de repetición y con un solo beam. El aviso de Transformers al enviar temperatura en modo greedy se conserva: ese parámetro se ignora. Véase la documentación de generación.

Cada generación se añade a `datos/resultados.csv`, con `parte=0` y costo cero. `datos/parte0_generaciones.jsonl` conserva además el prompt, los IDs, la semilla, la revisión del modelo, la fecha y los parámetros exactos. En esta parte `acierto` queda vacío porque no se evalúa exactitud.

Arriba se conservan las dos salidas del interruptor, las cinco corridas de `top_k=1` con sus IDs y soportes, la figura de cortes y la continuación de cien tokens sin editar. El conteo de cuatrigramas repetidos complementa la observación del texto; no evalúa su corrección.

Salida con `do_sample=False` (idéntica para $T = 0.2$ y $T = 1.5$):

```
calculate the total of all the items in the list.
```

```
$item = array( 'total' => $item, 'total' => $item, 'total'
=> $item, '
```

Con `do_sample=True` y `top_k=1`, las 20 corridas registradas en el CSV (4 ejecuciones del notebook con 5 semillas cada una) produjeron 1 salida distinta: idéntica a la de greedy.

Degeneración con 100 tokens en modo greedy (Holtzman et al. [1]), salida sin editar:

```
calculate the total of all the items in the list.
```

```
$item = array( 'total' => $item, 'total' => $item, 'total'
=> $item, 'total' => $item, 'total' => $item, 'total' =>
$item, 'total' => $item, 'total' => $item, 'total' =>
$item, 'total' => $item, 'total' => $item, 'total' => $
```

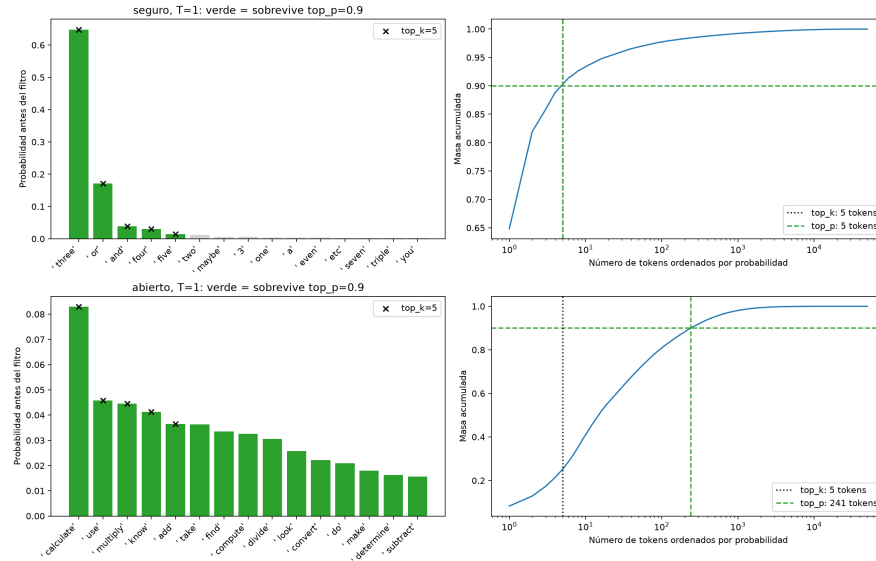


Figura 2. Dónde corta cada palanca: tokens que sobreviven a $\text{top_k}=5$ y a $\text{top_p}=0.9$ en cada prefijo. En el prefijo abierto los dos conjuntos no coinciden (5 frente a 241 tokens).

2.3 El límite del modelo base

Salida cruda de GPT-2 ante la instrucción exacta de las Partes 1 a 4:

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos

GPT-2 repitió partes del problema en lugar de responder con un número. Su entrenamiento está orientado a continuar texto. El ajuste con instrucciones puede mejorar ese comportamiento, aunque no garantiza respuestas correctas. También consideramos que el problema estaba en español y GPT-2 se entrenó principalmente en inglés.

3 Parte 1 — Comparación de modelos

Precios de la tabla semestral, cada uno con la fecha de verificación de su propia fila (Tabla 2).

Tabla 2. Parte 1: tres modelos sobre los 10 casos. Latencia = tiempo de pared de la petición.

modelo	exactitud	lat. (s)	tok. in	tok. out	costo USD	verificado
gpt-4o-mini	0	1.9063	793	12	0.00013	2026-08-26
gpt-5.6-luna	1	1.13	783	405	0.00064	2026-09-18
qwen3:1.7b	0.9	10.677	1011	7424	0	local

Elegiríamos gpt-5.6-luna: acertó los diez casos y costó aproximadamente 0,00064 dólares en total. Qwen acertó nueve sin cobro de API, pero tardó más y necesitó recursos locales. Ampliaríamos las pruebas antes de usar cualquiera en producción.

Confirmamos que los precios de la tabla se leyeron de la tabla semestral del enunciado, con la fecha de cada fila, y no de memoria.

4 Parte 2 — Decodificación

4.1 Matriz de exposición

Cada parámetro se mandó con dos valores extremos, 3 casos $\times$ 2 corridas. El estado se decide por la dispersión entre corridas de la tupla (salida, tokens de salida), no por la respuesta final (Tabla 3); los mensajes de error se transcriben en la Tabla 4.

Tabla 3. Parte 2.a: matriz de exposición, declarado frente a observado (2026-09-20). Evidencia: porcentaje de casos cuyas corridas difieren con cada valor; o número de llamadas con error.

modelo	parámetro	declarado (tabla)	observado	evidencia
gpt-4o-mini	temperature	si	aceptado y actual	temperature=0: 0% / temperature=1.5: 100%
gpt-4o-mini	top_p	si	aceptado y actual	top_p=0.1: 33% / top_p=1: 67%
gpt-4o-mini	top_k	no	rechazado	12/12 llamadas con error
gpt-5.6-luna	temperature	no_en_esta_fila	rechazado	12/12 llamadas con error
gpt-5.6-luna	top_p	no_en_esta_fila	rechazado	6/12 llamadas con error
gpt-5.6-luna	top_k	no	rechazado	12/12 llamadas con error
qwen3:1.7b	temperature	si	aceptado y actual	temperature=0: 33% / temperature=1.5: 100%
qwen3:1.7b	top_p	si	aceptado y actual	top_p=0.1: 33% / top_p=1: 100%
qwen3:1.7b	top_k	si	aceptado y actual	top_k=1: 33% / top_k=40: 100%

Tabla 4. Parte 2.a: mensajes de error transcritos de las celdas rechazadas.

modelo	parámetro	mensaje literal (truncado a 160 caracteres)
gpt-4o-mini	top_k	TypeError: Completions.create() got an unexpected keyword argument 'top_k'
gpt-5.6-luna	temperature	400: Error code: 400 - {'error': {'message': "Unsupported value: 'temperature' does not support 0.0 with this model. Only the default (1) value is supported.",
gpt-5.6-luna	top_p	400: Error code: 400 - {'error': {'message': "Unsupported parameter: 'top_p' is not supported with this model.", 'type': 'invalid_request_error', 'param': 'top_p
gpt-5.6-luna	top_k	TypeError: Completions.create() got an unexpected keyword argument 'top_k'

En gpt-4o-mini, temperatura y top-p mostraron cambios entre corridas. Top-k falló en el SDK, antes de llegar al servidor. Luna rechazó las temperaturas probadas y aceptó top_p=1, pero rechazó 0.1. En Ollama observamos contenido en thinking sin pedirlo, contrario al valor predeterminado indicado en la tabla.

4.2 Barrido de temperature y top-p

Estimación previa con el supuesto del enunciado (1000 tokens de entrada y 600 de salida por llamada) al precio de gpt-4o-mini verificado el 2026-08-26: 750 llamadas $\approx$ 0.38 USD. Se corrieron 450 (3 corridas por celda); costo real 0.03 USD.

Tabla 5. Parte 2.b: rejilla temperature $\times$ top_p sobre gpt-4o-mini, 3 corridas $\times$ 10 casos por celda.

T	top_p	exactitud	tokens out (media)	estabilidad
0.000	0.500	0.000	1.200	1.000
0.000	0.900	0.000	1.200	0.967
0.000	1.000	0.000	1.200	1.000
0.300	0.500	0.000	1.400	0.767
0.300	0.900	0.000	1.200	0.833
0.300	1.000	0.000	1.267	0.767
0.700	0.500	0.000	1.400	0.700
0.700	0.900	0.033	1.167	0.567
0.700	1.000	0.000	1.233	0.700
1.000	0.500	0.000	1.267	0.633
1.000	0.900	0.000	1.233	0.500
1.000	1.000	0.000	1.367	0.567
1.500	0.500	0.000	1.333	0.700
1.500	0.900	0.033	1.400	0.533
1.500	1.000	0.033	1284.133	0.367

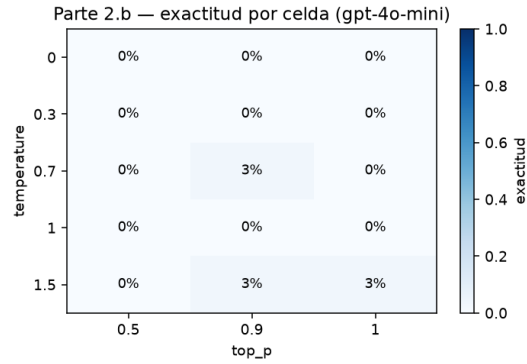


Figura 3. Exactitud por celda de la rejilla (gpt-4o-mini).

Tabla 6. Parte 2.b: barrido de top_k en local (qwen3:1.7b, T=1, 5 corridas secuenciales $\times$ 10 casos).

top_k	exactitud	tokens out (media)	estabilidad	casos idénticos x5
1.000	0.920	1080.320	0.960	8
5.000	0.960	947.780	0.880	7
40.000	0.980	931.120	0.860	7

Encontramos doce configuraciones con 0 % de exactitud y tres con 3,33 %. Ninguna fue suficiente para esta tarea. Usaríamos T=0 y top_p=1 como punto de partida por su estabilidad, pero tendríamos que mejorar el prompt o cambiar de modelo. Para titulares probaríamos T=1 y top_p=0.9, evaluándolos por separado. Al bajar la temperatura, la distribución concentra más probabilidad en los tokens favorecidos; al subirla, reparte más masa entre alternativas. En local, top_k=1 dio la misma salida en ocho de diez casos en cinco corridas secuenciales, no en los diez. Ejecutamos tres repeticiones por caso y configuración, no las cinco solicitadas.

5 Parte 3 — Prompting estructurado

Las cuatro plantillas están en `src/prompts.py`. Los ejemplos del few-shot son problemas inventados, no los diez casos. La variante estructurada usa el modo JSON del proveedor (`response_format` con tipo `json_object`), sin esquema estricto.

Tabla 7. Parte 3: cuatro variantes de prompting sobre gpt-4o-mini, 10 casos.

variante	exactitud	tokens in	tokens out	costo USD
zero_shot	0.1	793	11	0.00013
few_shot	0	1842	11	0.00028
cot	1	933	3160	0.00204
structured	0	883	61	0.00017

Zero-shot acertó un caso, few-shot ninguno y CoT los diez. CoT aumentó los tokens de salida de 11 a 3160 y el costo aproximado de 0,00013 a 0,00204 dólares. La variante estructurada produjo diez JSON válidos, pero ninguna respuesta correcta; usamos el modo JSON del proveedor, sin esquema estricto. Elegiríamos CoT para estos ejercicios, few-shot para mostrar criterios mediante ejemplos y salida estructurada cuando otro programa necesite leer la respuesta.

6 Parte 4 — Modelos de razonamiento y niveles de esfuerzo

6.1 Barrido de esfuerzo

Modelo gpt-5.6-luna. El dial es `reasoning_effort`. El contador de tokens de razonamiento es el campo `reasoning_tokens` que devuelve la API. Se corrió `low` completo antes de los otros dos: 287 tokens de razonamiento en 10 casos frente a los 1 500 supuestos por el enunciado.

Tabla 8. Parte 4.a: barrido de esfuerzo en gpt-5.6-luna, 10 casos por nivel.

esfuerzo	exactitud	tok. razonamiento	tok. visibles	lat. (s)	costo USD
low	1	287	101	1.2317	0.00062
medium	1	288	101	0.95785	0.00062
high	1	375	101	1.0125	0.00073

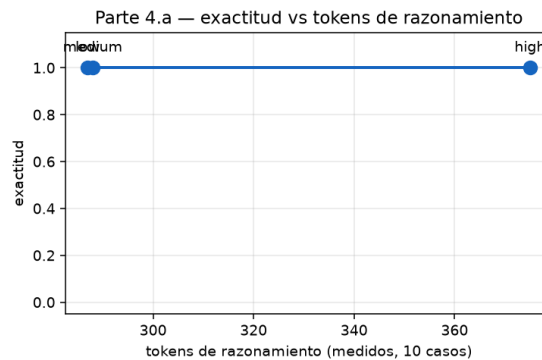


Figura 4. Exactitud frente a tokens de razonamiento medidos.

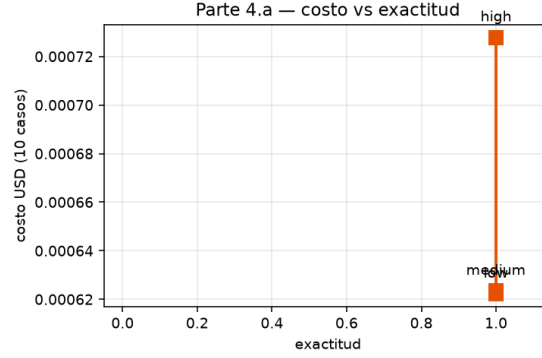


Figura 5. Costo en USD frente a exactitud.

Los tres niveles acertaron los diez casos. Low consumió 287 tokens de razonamiento; medium, 288; y high, 375. De medium a high pagamos 0,0001044 dólares adicionales sin mejorar la exactitud. Elegiríamos low para este conjunto. La similitud entre low y medium no demuestra que el parámetro se ignore; necesitaríamos más casos y repeticiones.

6.2 El caso donde pensar más hace daño

Casos c11 (distracción por lo irrelevante), c12 (sobreajuste al marco) y c13 (correlación espuria), según los modos de fallo de Gema et al. [2].

Tabla 9. Parte 4.b: los tres casos contaminados en los niveles extremos.

esfuerzo	caso	acierto	tokens razonamiento
high	c11	1	0
high	c12	1	24
high	c13	1	26
low	c11	1	0
low	c12	1	0
low	c13	1	21

No observamos empeoramiento: ambos niveles acertaron los tres casos. No reproducimos los fallos descritos por Gema et al., pero eso no contradice su estudio. Tres casos no permiten establecer una regla general.

7 Parte 5 — Reflexión teórica

7.1 Explica intuitivamente qué hace el mecanismo de atención en un transformer. ¿Por qué es importante el concepto de query, key, value?

Entendemos la atención como una forma de combinar información relevante del contexto. La query representa lo que una posición busca, las keys permiten comparar y los values aportan el contenido. En generación causal, el modelo no puede atender a tokens futuros.

7.2 ¿Cuál es la diferencia entre un modelo base y un modelo alineado? ¿Por qué un modelo base no es directamente útil para las aplicaciones de este curso?

Salida cruda de 0.c ante la instrucción de la tarea. GPT-2 entró en bucle y repitió este fragmento hasta el límite de tokens; se muestran las tres primeras repeticiones tal como salieron, sin editar:

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

Una cada una de 43 sillas. ¿Cuántos tornillos le sobran?

En nuestra prueba, GPT-2 continuó el enunciado sin resolverlo. El ajuste con instrucciones y preferencias busca mejorar la respuesta a peticiones. Para estas aplicaciones necesitamos seguir instrucciones y respetar formatos, pero también verificar el contenido.

7.3 En la Parte 3, la variante con Chain-of-Thought probablemente usó más tokens que la zero-shot. ¿Cuándo vale la pena pagar ese costo y cuándo no?

Pagaríamos CoT en estos ejercicios porque pasó de uno a diez aciertos. Para extraer un dato explícito, primero probaríamos una instrucción corta. Decidiríamos según la mejora de exactitud, el costo y la latencia medidos.

7.4 En tu matriz de la Parte 2.a hay al menos una celda «rechazado». ¿Qué gana un proveedor al retirar un parámetro de muestreo de su API, y qué pierde quien lo usaba?

Consideramos que retirar un parámetro puede simplificar el servicio y limitar configuraciones poco evaluadas. Quien lo usa pierde control sobre el muestreo. El esfuerzo ofrece otro ajuste, pero no sustituye directamente a la temperatura.

7.5 Los tokens de razonamiento de la Parte 4 se facturan y casi nunca se devuelven en crudo. Si tu modelo te mostrara la traza completa, ¿sería eso una explicación de cómo llegó a la respuesta?

No asumiríamos que una explicación convincente refleja cómo se obtuvo la respuesta. Lanham et al. encontraron que la influencia de la cadena cambia según la tarea y el modelo. Para auditar, combinaríamos las trazas con casos verificables y pruebas controladas.

8 Reproducibilidad y desviaciones declaradas

- Código, casos, `requirements.txt` y `datos/resultados.csv` (850 filas, una por llamada, las locales con costo cero) en <https://github.com/jastk45/Maestria->.
Carpeta: `IA-Generativa-y-Agentes/semana1/taller01_foundation_models`
- Las claves se leen de un `.env` excluido por `.gitignore`; ninguna aparece en el informe, el notebook ni el repositorio.
- La rejilla de 2.b se corrió con 3 corridas por celda en vez de 5. Las sondas de 2.a se mandaron con 4 peticiones en paralelo; el barrido de `top_k`, en secuencia.
- Los tokens de razonamiento de `qwen3` se cuentan como palabras del campo `thinking` de `/api/chat`; los de `gpt-5.6-luna`, del contador de la API.

References

1. Holtzman, A., Buys, J., Du, L., Forbes, M., Choi, Y.: The Curious Case of Neural Text Degeneration. ICLR (2020). <https://arxiv.org/abs/1904.09751>
2. Gema, A.P., et al.: Inverse Scaling in Test-Time Compute. TMLR (2025)
3. Lanham, T., et al.: Measuring Faithfulness in Chain-of-Thought Reasoning. arXiv:2307.13702 (2023)
4. Vaswani, A., et al.: Attention Is All You Need. NeurIPS (2017)
5. Radford, A., et al.: Language Models are Unsupervised Multitask Learners. OpenAI (2019). Model card: <https://huggingface.co/openai-community/gpt2>